

ADR 0020 — Idempotent operation lifecycles and reconnectable progress

- **Status:** Accepted
- **Date:** 2026-07-25
- **Milestone / issue:** M1.08 — Add idempotent operation lifecycles and progress (#61)
- **Supersedes / superseded by:** — (attaches to ADR 0016's single funnel and ADR 0019's per-repository guard; makes ADR 0018's staleness gate answerable instead of merely protective)

Context

Every mutation already funnels through one chokepoint (ADR 0016) and one repository guard (ADR 0019). What it still lacked was **identity over time**. A request was anonymous: no name, no recorded state, no existence outside the TCP connection carrying it. Three consequences, all felt hardest on the iPad-over-SSH-tunnel path this project is built for:

1. A dropped connection cancelled the git command mid-flight — axum drops the handler future when the client disconnects.
2. A retry was indistinguishable from a new intent, so the user tapping Commit again after a dead spinner risked a second commit.
3. The outcome was unrecoverable once the response was lost. There was no "what happened to that commit?" endpoint.

ADR 0018's staleness gate blunted (2) — a second commit is refused because the generation moved — but a refusal is not an answer. The user wanted to know whether their commit landed, and a 409 doesn't say.

Decision

Give every mutation a durable identity, a lifecycle, and a replayable result, and let the client name its intent with an **idempotency key** so a retry is recognised as a retry.

Detached run, synchronous wait

The POST no longer *is* the execution — it *observes* one:

```

POST /api/commit    x-git-vista-idempotency-key: <key>
|
├─ registry.admit(key, operation)          → Accepted    (new record, id minted)
|   └─ tokio::spawn(pipeline)              → Running      (detached: survives disconnect)
|                                           → Succeeded | Failed
|
└─ await record.wait_terminal()            → replay the recorded (status, body)

```

The response body and status stay byte-identical to before — git's own message, forwarded verbatim — with the operation id added as a response header (`x-git-vista-operation`). No existing endpoint's contract changes shape.

A second POST carrying the same key finds the record and never plans, never locks, never runs git: it awaits the in-flight one, or replays the terminal result.

Rejected alternative — **202 Accepted, fully async**: textbook, but it changes every write endpoint's contract at once and forces pending-state rendering into a frontend whose state refactor is a later issue. The detached-run model buys the same reconnectability at a fraction of the blast radius.

Idempotency keys bind to the operation, not just the name

The registry stores the plan's `operation_hash` (ADR 0015) beside the record and refuses a key reused with a different operation with **409 Conflict**. A key alone is a footgun: without the hash check, the same key with a different body would replay a result computed for something else.

A key is minted by the client **per user action**, not per HTTP attempt — the frontend mints one in `new_idempotency_key()` and reuses it across the existing network-failure retry. A genuine double-tap is two keys, two operations, and the second is refused by the staleness gate exactly as before; only a retry of the same attempt replays.

Lifecycle states

```

Accepted → Running → Succeeded
           ↳ Failed

```

Refusals that happen before admission (read-only mode, malformed body, missing key, protocol, CSRF) stay plain synchronous 4xx — they never become operations, because nothing was ever attempted.

Terminal results carry what the client needs to reconcile

Field	Source
state , stage	the lifecycle
status , message	the recorded response, replayed verbatim
generation	recomputed from the live repository <i>after</i> execution
recovery	the plan's typed RecoveryStrategy (ADR 0015)
operation_hash	binds the record to one exact operation
repository , worktree	opaque tokens, never paths (ADR 0003)
accepted_at , ended_at	Unix seconds, server clock

The post-execution generation is the datum that lets a reconnecting client decide whether its cached graph is stale without re-reading everything.

Progress: server-sent events, bounded and authenticated

`GET /api/operations/{id}/events` streams lifecycle and stage transitions and closes on the terminal state. Authenticated by the same session cookie every other route requires.

Protocol negotiation via query string, for this one route. `EventSource` cannot set request headers, so this route accepts `?protocol=<n>` and validates it with the *same* `check_compatibility` code the header path uses. Matched structurally in `middleware:accepts_protocol_query` (path prefix + suffix + segment count) so the exception cannot widen when a sibling route is added later.

Bounded four ways: closes at the terminal event; a 15s heartbeat comment; a 30-minute lifetime cap; and a process-wide cap on concurrent live streams (`operations::MAX_LIVE_STREAMS = 32`), so a client that opens and abandons streams cannot exhaust the server.

Stage events (`queued` , `planning` , `waiting` , `checking` , `executing` , `finished`) are the planner's real stages, not invented UI steps — a stuck operation names the thing it is stuck on (`waiting` means another mutation of this repository holds the ADR 0019 guard). Streaming git's own stderr line by line is real work and is deliberately not built here.

The registry is in-memory, by design and for now

Bounded to the newest 256 records with a 1-hour TTL, and a record that is not terminal is **never** evicted — dropping a live record would strand the request awaiting it. Persistence is a future issue; `OperationStatus` is shaped to be the thing it would write to disk. A server restart forgets in-flight operations — the same consequence ADR 0019 already accepts: the client re-POSTs, the generation has moved, and it is told so.

Protocol version 2 → 3

New request header (`x-git-vista-idempotency-key`), new response header (`x-git-vista-operation`), new routes. An old cached PWA client would send no key and could not name its operation, so the window moves to [3, 3] rather than tolerating a client silently missing the replay guarantee. Precedent: ADR 0013.

The idempotency header is **required** on writes, enforced at the planner funnel (`plan_and_execute`) rather than a middleware route list — a route list would drift the first time a handler is added; the funnel cannot, by construction (ADR 0016).

Alternatives considered

- **202 Accepted / fully async everywhere.** See above — correct in the abstract, too large a blast radius for the frontend state this project has today.
- **Minting a key server-side when the client omits one.** Rejected: it would silently give an unmigrated client none of the guarantees this issue exists to provide. A missing key is refused instead.
- **Keying the registry by `OperationId` only, no hash check.** Rejected — the hash check is what makes an idempotency key trustworthy rather than a way to get a confidently wrong answer.
- **Route list for the header requirement.** Rejected in favor of the funnel check, matching ADR 0016's own reasoning for why the planner is the single place a mutation can begin.

Consequences

1. **A dropped tunnel stops being data loss.** The pipeline runs to completion in a detached task regardless of the request future's fate.
2. **A retry is safe by construction**, everywhere — not just on the one endpoint (`/api/branch`) that used to have a hand-rolled retry because a duplicate happened to be harmless there. `git-vista/src/api.rs`'s `send_write` retry now works for every write, because both attempts carry the same key.
3. **The client can always ask "what happened?"** via `GET /api/operations/{id}`, independent of the stream.
4. **The registry is process-global, in-memory state** — the one new stateful component in the server, sitting beside `coordinator`'s guard map and `session`'s session store. Same discipline: a `std::sync::Mutex` never held across an `.await`.
5. **`ErrorCode` has no dedicated `Conflict` variant** — the 409 from a reused-key/different-operation refusal reports as `bad_request` (`ErrorCode::from_status`'s existing fallback for unmapped 4xx). Pre-existing gap (ADR 0018's staleness 409s hit the same fallback); left as-is here. Adding a variant is itself a wire-contract decision, out of scope for this issue.
6. **`docs/adr/0016 - 0019`'s PDF twins are still tracked in git**, which violates the project's own "ADR `.md` tracked, PDF untracked" rule (`.gitignore` now excludes `docs/adr/*.pdf` going forward, but the already-committed ones were left for a human decision rather than `git rm --cached d` as part of this change).

Where this is implemented

- `crates/git-vista-protocol/src/operation.rs` — `IdempotencyKey`, `OperationId`, `OperationState`, `OperationStage`, `OperationStatus`, `ProgressEvent`.
- `crates/git-vista-protocol/src/newtype.rs` — the validating-newtype machinery shared with `plan.rs` (`require_token`, the `validated_string!` macro), extracted so both modules enforce one definition of each rule.
- `crates/git-vista-protocol/src/version.rs` — `PROTOCOL_VERSION = 3`, `IDEMPOTENCY_HEADER`, `OPERATION_HEADER`, `PROTOCOL_QUERY`.
- `crates/git-vista-server/src/operations.rs` — the registry: `admit`, `Record`, `OperationHandle`, eviction, the task-local idempotency/progress scopes, `StreamPermit`.
- `crates/git-vista-server/src/planner.rs` — `plan_and_execute` requires the key; `plan_and_execute_tracked` admits, spawns the pipeline detached, and awaits the terminal result; stage reports (`planning` / `waiting` / `checking` / `executing`) are woven into `plan_and_execute_in`.
- `crates/git-vista-server/src/handlers/operations.rs` — `GET /api/operations/{id}` and the SSE stream.
- `crates/git-vista-server/src/middleware.rs` — the query-string protocol exception, and the idempotency layer that scopes the key and stamps the minted id onto the response.
- `crates/git-vista/src/api.rs` — `new_idempotency_key`, `send_write` (mints once, retries under the same key on network failure).
- `crates/git-vista-server/src/planner/lifecycle_suite.rs` — the acceptance tests: recorded outcome shape, retry-runs-git-once, late replay, key conflict, disconnect survival, fetch-by-id.
- `crates/git-vista-server/src/middleware.rs (mod tests)` and `crates/git-vista-server/src/handlers/operations.rs (mod tests)` — wire- level negotiation, idempotency-header validation, and SSE handler tests.

Signed: thomas2010 · 2026-07-25T08:20:00-04:00